Python Engineering

WEEK 03 - Modules & Coordinates | CLASS 10 - Advanced Functions | EX18

Objetivo: aprender argumentos posicionales, keyword, default, *args, **kwargs, unpacking y funciones pasadas como argumentos.

Valores de referencia para EX18

Usaremos estos valores para que cada bloque tenga una referencia clara. Para demostrar el override de gravedad usamos h0 = 100 m: si h0 fuera 0 m, el termino g*h0 seria cero y cambiar g no modificaria la energia inicial.

Variable / caso	Valor de referencia	Uso
h0	100.0 m	Permite ver el efecto de cambiar g
v0	50.0 m/s	Velocidad inicial
g default	9.81 m/s^2	Valor por defecto en initial_energy()
g override	1.62 m/s^2	Ejemplo de sustitucion del valor por defecto
measurements	[10.2, 10.5, 9.9, 10.1]	Unpacking con *
summary	129.9 m, 10.3 s, -51.0 m/s	**kwargs
x, y	5, 3	Funcion como argumento

Resultados esperados principales

```
initial_energy(100.0, 50.0)
# g = 9.81 -> 2231.0 J/kg

initial_energy(100.0, 50.0, g=1.62)
# override g -> 1412.0 J/kg

average_measurements(*[10.2, 10.5, 9.9, 10.1])
# 10.175

apply_operation(add, 5, 3)
# 8

apply_operation(multiply, 5, 3)
# 15
```

BLOCK A - Positional, keyword y default arguments

```
def initial_energy(h0, v0, g=9.81):
    return g * h0 + 0.5 * v0**2

initial_energy(100.0, 50.0)
initial_energy(h0=100.0, v0=50.0)
initial_energy(100.0, 50.0, g=1.62)
```

Compara la llamada posicional, la llamada keyword y el override del valor por defecto.

BLOCK B - *args

```
def average_measurements(*values):
    return sum(values) / len(values)

measurements = [10.2, 10.5, 9.9, 10.1]
average_measurements(*measurements)
```

BLOCK C - **kwargs

```
def simulation_summary(**kwargs):
    for key, value in kwargs.items():
        print(key, value)

simulation_summary(
    max_altitude=129.9,
    flight_time=10.3,
    final_velocity=-51.0
)
```

BLOCK D - Dictionary unpacking con **

```
config = {'h0':100.0, 'v0':50.0, 'g':9.81}
initial_energy(**config)
```

BLOCK E - Funcion como argumento

```
def add(a, b):
    return a + b

def multiply(a, b):
    return a * b

def apply_operation(operation, x, y):
    return operation(x, y)

apply_operation(add, 5, 3)
apply_operation(multiply, 5, 3)
```

VALIDACION FINAL

```
Explain in comments:
- Difference between positional and keyword arguments
- What a default argument does
- What *args receives and its type
- What **kwargs receives and its type
- Difference between * when defining and * when calling
- Difference between ** when defining and ** when calling
- Why apply_operation(add, 5, 3) passes add without ()
```

Criterio de cierre

EX18 estara completado cuando puedas elegir conscientemente que tipo de argumento conviene usar y puedas pasar una funcion a otra funcion entendiendo que la funcion tambien puede tratarse como un objeto.